

Data Engineering Assessment Handoff

StackUp Engineering Academy | Adil Razack | Status verified 23 September 2026

Overall outcome: The four pillars are implemented in the repository. The PostgreSQL warehouse schema has also been created successfully in the running Docker database. The six tables currently exist but contain zero rows.

1. Pillar Outcomes

Pillar	Outcome	Main locations
1. Foundations	Project and employee cleaning, star-schema DDL, SCD Type 2 employee model, and employee data-quality work are implemented.	solutions/submissions/adil_razack/01_foundations/ outputs/results/adil_razack/01_foundations/
2. SQL and Visualization	Transaction ETL, six business queries, optimisation notes, cleaned CSV outputs, and dashboard/mockup artifacts are present.	solutions/submissions/adil_razack/02_sql_and_viz/ outputs/results/adil_razack/02_sql_and_viz/ presentation/
3. Big Data	Spark event processing, Kafka producer/consumer logic, and Airflow orchestration with a DQ gate are implemented. Spark outputs include Parquet summaries.	solutions/submissions/adil_razack/03_big_data/ outputs/results/adil_razack/03_big_data/
4. Infrastructure and Governance	Docker support, configurable DQ framework, governance documentation, retention, PII, and access guidance are present.	solutions/submissions/adil_razack/04_infrastructure/ outputs/results/adil_razack/04_infrastructure/ docker-compose.yml

2. Key Deliverables

- Warehouse model: [solutions/submissions/adil_razack/01_foundations/data_model.sql](#)
- Project ETL output: [outputs/results/adil_razack/01_foundations/projects_clean.csv](#)
- Employee ETL output: [outputs/results/adil_razack/01_foundations/employees_clean.csv](#)

- Transaction ETL: `solutions/submissions/adil_razack/02_sql_and_viz/etl_full.py`
- Transaction output: `outputs/results/adil_razack/02_sql_and_viz/transactions_clean.csv`
- SQL queries: `solutions/submissions/adil_razack/02_sql_and_viz/queries.sql`
- Dashboard artifacts: `presentation/pillar2_powerbi_report.html`,
`presentation/pillar2_dashboard_mockup.html`, and `presentation/pillar2_powerbi_dataset.csv`
- Governance: `solutions/submissions/adil_razack/04_infrastructure/data_governance.md`

3. Warehouse Status

The schema was executed successfully against PostgreSQL in Docker using database `airflow`.

Connection	Value
Host and port	localhost:5432
Database	airflow
User	presight
Tables	dim_date, dim_project, dim_vendor, dim_employee, bridge_employee_project, fact_transactions
Current rows	Zero in each table. DDL exists; data loading is not part of the completed run.

Important: The assessment requires cleaned CSV outputs and SQL/schema work. A populated live warehouse is useful for demonstration, but it is a separate runtime loading step and was not completed here.

4. How to Re-run

Run PowerShell from the repository root. The repository path is:

```
C:\Users\adil.razack\OneDrive - Presight AI\Documents\GitHub\stackup-engineering-academy_assessment
```

Or navigate using:

```
Set-Location "C:\Users\adil.razack\OneDrive - Presight AI\Documents\GitHub\stackup-engineering-academy_assessment"
```

Run individual pillars

```
.\scripts\run_pillar_1.ps1
.\scripts\run_pillar_2.ps1
.\scripts\run_pillar_3.ps1
.\scripts\run_pillar_4.ps1
```

Run all pillars

```
.\scripts\run_all_pillars.ps1
```

Start Docker services

```
docker compose up -d
docker compose ps
```

Services: PostgreSQL on localhost:5432, Kafka UI on http://localhost:8080, and Airflow on http://localhost:8081.

Run ETL directly

```
python .\solutions\submissions\adil_razack\01_foundations\etl_pipeline.py
python .\solutions\submissions\adil_razack\02_sql_and_viz\etl_full.py
python .\solutions\submissions\adil_razack\04_infrastructure\dq_framework.py
```

5. Pillar 3: Run and Show Outputs

Code: solutions/submissions/adil_razack/03_big_data/

Run everything in Pillar 3:

```
.\scripts\run_pillar_3.ps1
```

This runs Spark first and then Kafka producer/consumer processing. Kafka requires Docker services to be running:

```
docker compose up -d
$repo = (Get-Location).Path
docker run --rm `
    --name presight-spark `
    -v "${repo}:/opt/project" `
    -w /opt/project `
    apache/spark:4.0.1-python3 `
    /opt/spark/bin/spark-submit `
    solutions/submissions/adil_razack/03_big_data/spark_pipeline.py
python .\solutions\submissions\adil_razack\03_big_data\kafka_streaming.py --mode both --delay-seconds 0.001
```

Docker runs Spark on Linux inside the container, so Windows does not need winutils.exe or HADOOP_HOME.

Expected Spark outputs: five Parquet directories under:

```
outputs\results\adil_razack\03_big_data\spark\project_activity_summary
outputs\results\adil_razack\03_big_data\spark\user_activity_summary
outputs\results\adil_razack\03_big_data\spark\escalation_log
```

```
outputs\results\adil_razack\03_big_data\spark\daily_event_volume
outputs\results\adil_razack\03_big_data\spark\peak_usage_analysis
```

Each directory contains a part-*.parquet data file and _SUCCESS marker. To display a Parquet result:

```
python -c "from pathlib import Path; import pandas as pd;
p=next(Path('outputs/results/adil_razack/03_big_data/spark/project_activity_summary').glob('part-*.parquet')); print(pd.read_parquet(p).head(20).to_string(index=False))"
```

Expected Kafka output:

```
outputs\results\adil_razack\03_big_data\kafka\summary.json
```

Display the Kafka summary:

```
Get-Content .\outputs\results\adil_razack\03_big_data\kafka\summary.json
```

Kafka topics can be inspected in Kafka UI at <http://localhost:8080>. The Airflow DAG is available at <http://localhost:8081>; log in with admin / admin, enable the DAG, trigger it, and open task logs to show the DQ gate and XCom results.

6. Pillar 4: Run and Show Outputs

Code and configuration:

```
solutions\submissions\adil_razack\04_infrastructure\dq_framework.py
solutions\submissions\adil_razack\04_infrastructure\dq_config.yml
solutions\submissions\adil_razack\04_infrastructure\data_governance.md
```

Run the DQ framework:

```
.\scripts\run_pillar_4.ps1
or
python .\solutions\submissions\adil_razack\04_infrastructure\dq_framework.py
```

The framework runs six checks for each configured dataset: completeness, uniqueness, numeric validity, date validity, consistency, and referential integrity.

Expected output reports:

```
outputs\results\adil_razack\04_infrastructure\dq_report_projects.md
outputs\results\adil_razack\04_infrastructure\dq_report_employees.md
outputs\results\adil_razack\04_infrastructure\dq_report_transactions.md
outputs\results\adil_razack\04_infrastructure\dq_report_employees_salary_history.md
```

Display reports in PowerShell:

```
Get-Content .\outputs\results\adil_razack\04_infrastructure\dq_report_projects.md
```

```
Get-Content .\outputs\results\adil_razack\04_infrastructure\dq_report_transactions.md
```

Open the governance document in VS Code:

```
code .\solutions\submissions\adil_razack\04_infrastructure\data_governance.md
```

7. Warehouse Schema Commands

Check the tables

```
docker exec presight-postgres psql -U presight -d airflow -c "\dt"
```

Recreate the schema only when needed

Do not rerun this against an existing populated database unless you intentionally drop the tables first.

```
Get-Content -Raw ".\solutions\submissions\adil_razack\01_foundations\data_model.sql" | docker  
exec -i presight-postgres psql -v ON_ERROR_STOP=1 -U presight -d airflow
```

Check row counts

```
docker exec presight-postgres psql -U presight -d airflow -c "SELECT 'dim_date' AS table_name,  
COUNT(*) FROM dim_date UNION ALL SELECT 'dim_project', COUNT(*) FROM dim_project UNION ALL  
SELECT 'dim_vendor', COUNT(*) FROM dim_vendor UNION ALL SELECT 'dim_employee', COUNT(*) FROM  
dim_employee UNION ALL SELECT 'bridge_employee_project', COUNT(*) FROM bridge_employee_project  
UNION ALL SELECT 'fact_transactions', COUNT(*) FROM fact_transactions;"
```

8. Assessment Notes and Limitations

- The Docker Compose warning about the obsolete version attribute is non-blocking. The services started successfully.
- A native Power BI .pbix file cannot be generated without Power BI Desktop. HTML and CSV reporting substitutes are available in presentation/.
- The schema and table creation are verified. The fact and dimension tables are empty until a staging/load process is run.
- The existing ETL produces cleaned CSV staging outputs, not separate dim_*.csv or fact_transactions.csv files. This matches the Pillar 2 assessment requirement for three cleaned DataFrame CSVs.

9. Final Handoff Checklist

Item	Status
Four pillar solution code present	Complete
Cleaned CSV and report artifacts present	Complete
Docker services available	Verified running
PostgreSQL schema materialized	Verified, six tables
Warehouse data loaded	Pending, tables are empty

Native Power BI PBIX	Not available; environment limitation
----------------------	---------------------------------------